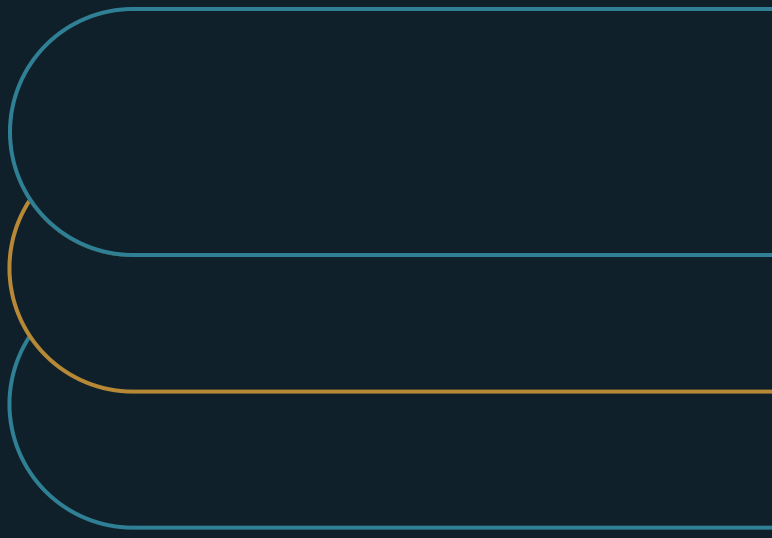
11

WHITE PAPER · AI OPERATING GOVERNANCE

The Demo-to-Production Gap

A demonstration is not operating evidence



Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

Scale on operating evidence, not demo impressions. Rely on exactly what the evidence covers.

A demo proves a system can produce a nice result under chosen conditions. Production needs proof that a real workflow can lean on it, over and over. This paper specifies that proof as seven production questions, asked before scale - and shows how to write the one-page reliance decision that answers them.

01 A pilot with curated cases and enthusiast users is a longer demo. Selection effects stay intact until real inputs arrive in real condition.

02 A weak answer to any production question is a boundary rather than a veto: rely on the system for exactly what the evidence covers.

03 Fund the exception path, the support model, and the stop mechanism at approval time - in the same document as the benefit case.

CONTENTS

The argument

A demo proves one thing: the system can produce a nice result under chosen conditions. Production needs more. It needs proof that a real workflow can lean on the system, over and over. That means owners, hard cases, exceptions, controls, support, and a way to stop.

I have sat through plenty of demos built to sell a room. Clean inputs go in, an impressive result comes out, and the room is sold in minutes. What the room never sees is the input set: a handful of cases

picked from thousands, in the formats the system handles best, with none of the messy ones that make up a third of the real work. The gap between the demo and the real repository is the entire operating problem, and it was edited out before anyone entered the room.

There is nothing dishonest about this. Demos exist to show what is possible. Chosen inputs, skilled operators, and rehearsed paths are the genre. The governance failure happens afterward, when possibility gets quietly promoted to proof — when a system that performed under selection is authorized to perform under ordinary users, ordinary data, and ordinary pressure, with no one asking what changed between those two worlds.

Enterprise surveys keep finding the same shape at scale: broad experimentation, much narrower operating value [1][2]. This paper argues the gap is closed by different evidence, not by better demos or bigger pilots. It specifies that evidence as seven production questions, asked before scale. The questions come from my own portfolio and AI-governance work. The paper is written for AI investment owners, product and technology leaders, workflow owners, and risk executives.

What a demo actually proves

A demo is proof of existence. Under some conditions, this system produces this result. That is real information. It rules out impossibility, which early decisions legitimately need. The trouble is that everything that makes a workflow production-worthy is exactly what an existence proof cannot address: behavior across all the real inputs, performance under users who did not build the system, what happens when a dependency is down, and the cost of ownership after the launch team leaves.

Selection operates on more than inputs. The demo's operator knows which phrasings work. The pilot's users got training the eventual population will skim. The demo environment has no permission boundaries, no data-quality debt, no quarter-end load. Each removed friction is a small loan against production, and the loans come due together, in the first month of ordinary use. That is why so many systems "worked in the pilot and struggled at scale." Scale did not break them; it simply repaid the selection.

Pilots inherit the problem more subtly. A pilot with build-team operators, curated cases, and enthusiast users is a longer demo. It generates real usage data, which makes it feel like production evidence. But the selection effects are intact: the pilot population volunteered, the hard cases were deferred, and the engineers watching the queue handled the exceptions by hand. Scaling decisions made on pilot enthusiasm often measure the team's willingness to make the system work — not the system's ability to work.

This is precision about what question each artifact answers. A demo answers "is this possible?" A pilot answers "can motivated people make this useful?" Approving production use needs a third question: "can this workflow rely on it under ordinary conditions?" That question has seven parts.

Why does the promotion from possibility to proof happen so easily? Three forces, none technical. Procurement momentum: by the time the demo lands, budget and sponsorship want a reason, and the demo supplies one. Asymmetric visibility: the demo's success is vivid and shared in a room, while production failures are statistical and belong to the future. And incentives: the champion who found the system is rewarded for the win, while the boundary conditions bring no reward. A production gate exists to give the boundary conditions a constituency. By default they have none.

The seven production questions

Each question names a category of evidence that demos cannot supply. Together they form the gate between experimental success and operating reliance. A weak answer to any one is a boundary rather than a veto: the reliance decision shrinks to whatever the evidence actually covers.

EXHIBIT 1

The production readiness path

01

Workflow fit

Did the complete job improve, or just the automated step?

02

Accountable owner

Who answers for the consequence when the output is wrong?

03

Representative evidence

Was it tested on the real population, including the hostile cases?

04

Exception and control

Can failures recover and can boundaries prove they were enforced?

05

Lifecycle and stop

Who funds support and can stop it with a tested fallback?

Workflow fit

The first question moves from the task to the whole job: production evidence starts with the complete path. Where do inputs come from, and in what condition? Which decisions consume the output? Who hands off to whom? What timing does the operation actually need? A system can excel at its task while

the workflow around it absorbs the savings — users reconciling systems, rewriting output into house format, chasing approvals the automation cannot request.

The evidence is a mapped end-to-end path with measurements at the joints: cycle time for the whole job, touches per case, rework loops, queue time between steps. The test is blunt. Did the complete job improve, or did the automated step improve while the job stayed the same length? Proceed only when the whole job moves, without shifting unacceptable burden into someone else's column.

A concrete shape of the failure: an extraction step gets fast, but the job is "obligations tracked and renewals actioned." Between the two sit intake scanning, a legal review that trusts nothing it has not seen, a system that needs the output re-keyed, and a renewals team on a different calendar. Automating the extraction without touching those joints produces a faster input to the same bottlenecks. Visible in any end-to-end measurement. Invisible in the demo's ninety seconds.

Operating ownership

The second question asks who answers for the consequence. A model owner is not a process owner. When the system misses an obligation and a renewal lapses, the harm lands in a business process, and only a business owner can accept that risk, define the reliance boundary, and decide what happens next. Pilots run fine without this clarity because the build team fills every gap by hand. Production cannot.

The evidence is named acceptance: a process owner who signed the reliance boundary, technical and data owners with service expectations, a risk owner for the controls, and support ownership with real capacity. The key artifact is the process owner's decision record: what the workflow may rely on the system for, what stays human, and what evidence would change either. Hold production use until the receiving operation owns both the benefit and the failure response. A system whose harms have no owner is loose in the operation rather than deployed into it.

Ownership is also where vendor relationships get honest. A vendor demo implies the vendor's confidence. A reliance decision requires the buyer's. The process owner who must sign the boundary suddenly has questions procurement never asked: what happens to our cases when your model updates, what does your exception rate look like on inputs shaped like ours, who do we call at 2 a.m. Those questions, asked before signature, are the cheapest due diligence available. The vendor's comfort answering them is itself evidence.

Representative cases

The third question attacks the demo's central edit: the input set. Happy-path accuracy says little about the rare, unclear, stale, and half-complete inputs that eat most operating judgment. Build the test set

from production history, plus deliberate hostility. The amendment chains. The scanned pages. The cases users mishandle. The inputs a rushed or careless user might actually submit.

The evidence is a case set with provenance: where each class came from, how frequencies were chosen, results by subgroup, and failure analysis weighted by severity rather than one accuracy number. Just as important is the boundary statement — which case population the results actually represent. The reliance decision then has a natural shape: permit use on the tested population, exclude what was not tested, and expand the boundary only as the case set expands. Untested means out of bounds, not approved by silence.

Building the hostile case set is faster than teams expect, because the operation already knows its monsters. An afternoon with the people who do the work today yields the list: the formats that break things, the cases that get argued about, the inputs that arrive half-complete every quarter-end. Add a sample from real history with its natural mess intact, and the set exists. What it costs is nerve — the willingness to score the system on the cases it was hoping to avoid.

Exception recovery

The fourth question assumes failure and asks what happens next. Production readiness includes detection (the workflow discovers quickly that a case fell outside the rules), routing (the case reaches someone with context and authority), intervention (a human can act before harm spreads), rollback (partial actions can be unwound), and return to a known state. A pilot celebrates average performance. An operation lives with its worst cases.

Only operating proof counts here, not architecture. How fast are failures detected? How long did recovery take on rollbacks that actually ran? How deep and how old is the exception queue? What does the exception path cost to staff at real volume? That last number quietly decides the case. If exceptions eat more specialist hours than the automation saves, the business case has flipped, whatever the accuracy is.

Design the exception path as a workflow, not a bucket. Someone specific receives the case, with context attached — what came in, what the system attempted, why it stopped — and a service expectation that matches the consequence. The anti-pattern is the generic manual-review pile, which absorbs failures until an aging backlog becomes its own incident. If the pilot never fed the exception path real cases, the path has not been tested. It has been drawn.

Observable controls

The fifth question moves boundaries from policy to proof. A rule that sensitive data stays out of prompts, or that big actions need approval, is just a sentence — until the system can show when the

boundary was approached, crossed, or enforced. Production reliance needs controls that leave evidence as they run. Logs tied to users and cases. Review records. Alerts that fired and were handled. Regular tests that the control still works.

Insist on the difference between a control that exists and a control you can inspect. A control that merely exists survives a design review; surviving an incident, an audit, or a customer's due-diligence checklist takes one you can inspect. The rule cuts both ways: shrink the system's authority to what its controls can show, and grow authority only as visibility grows with it.

Visible controls also protect the system's advocates. Some quarter, someone will question the system — an incident, an audit, a new executive. The difference between a defensible system and a doomed one is whether its owners can produce the operating record. Systems with inspectable histories survive scrutiny; assurances alone do not, however well the system was actually behaving.

Lifecycle support

The sixth question prices the years after launch. Models change on vendor schedules. Prompts and retrieval data drift. Policies move. Users rotate. A demo has no lifecycle. A pilot's lifecycle is the engineers' attention. Production needs funded, owned, standing capacity: evaluation maintenance, vendor-change response, user support, incident response, and regular re-testing.

The evidence is a support model with numbers in it: expected ticket volume, evaluation cadence and its owner, vendor-notice handling, budgeted hours, and named successors for the knowledge living in the build team's heads. The launch-team-disbands pattern is the most reliable predictor of quiet degradation a year later. Do not scale a system whose continuing obligations are unfunded. The obligation does not disappear. It transfers to whoever notices last.

Budget the lifecycle at approval time, in the same document as the benefit case, because that is the only moment the numbers compete fairly. A system that saves four thousand hours a year and needs twelve hundred in standing support is still worth running. But the twelve hundred should be visible while the four thousand is being celebrated — not discovered next year as an unfunded surprise.

Stop authority

The seventh question is the one companies most reliably skip: who can turn it off, on what evidence, with what fallback? Without a predefined stop, weak performance persists by default. Every threshold gets debated after the fact. Every pause proposal fights adoption momentum and sunk cost. Continued operation becomes the path of least resistance, whatever the numbers say.

The evidence is a stop design that has been exercised: named stop conditions with thresholds, a responsible owner, a communication path, a fallback that has actually run — the manual procedure still

works, the team still knows it — and re-entry criteria stating what evidence restores service. The rule: stop or narrow when the boundary is crossed, regardless of momentum. An untested fallback is a hope. A tested one is why stopping is politically possible at all.

Stop authority changes behavior even when it is never used. A build team that knows the system can be paused designs for pausability: cleaner state, better logging, reversible actions. An operating team that trusts the stop mechanism reports anomalies earlier, because raising a hand no longer means owning a crisis alone.

The questions as a gate

Together the seven questions replace the demo's implicit question — "is this impressive?" — with the operating one: what may this workflow rely on, within what boundary, owned by whom, watched how, supported by what, and stoppable when? The gate's output is a reliance decision with those clauses filled in, not a plain yes or no. Written that way, the decision is inspectable, revisitable, and sized to the evidence: a small reliance for thin evidence, a wider one as evidence accumulates.

The questions interlock. Representative cases feed exception design. Exception costs feed lifecycle pricing. Observable controls make stop conditions enforceable. Ownership makes every other answer accountable. In practice the gate fails at its weakest clause, which is why partial credit is dangerous: six strong answers and no stop authority is a system that will run until its first bad month becomes a bad quarter.

Replacing the demo approval

The gate installs as a replacement for the demo-driven decision, not an addition to it:

- Replace the demo approval with a production-evidence plan.
- Pilot inside the complete workflow with representative users and cases.
- Fund exception handling, evaluation, support, and incident response as standing work.
- Require a reliance decision that states permitted use, boundaries, controls, owner, and stop conditions.

The reliance decision deserves a standard shape, one page. What uses are permitted, and on which tested cases they rest. What is excluded. Who owns what. Which controls exist, and where their evidence lives. The support model and its budget line. The stop conditions and the fallback. And what evidence would expand or shrink any clause. One page works precisely because the seven questions were answered before the page was written.

Running the pilot inside the real workflow is the highest-leverage change. It turns the pilot from a longer demo into an evidence engine. Real inputs arrive in real condition. Real users generate the edge cases. The exception path gets exercised by reality instead of test design. And if the pilot cannot run inside the real workflow for risk reasons, that fact is itself evidence of how much reliance the system has earned so far.

The gate needs an owner with standing — usually the AI program function, jointly with risk. It also needs a service promise. Evidence plans get reviewed within a stated window. Small, reversible requests ride a fast track. The gate's authority should come from the reliance decisions it enables, not the approvals it withholds.

Expect the gate to slow the enthusiasm curve, and treat that as the feature it is. Demo-driven cadence promotes systems at the speed of impressions. The gate promotes them at the speed of evidence. The deals that survive are slower and durable. The ones that die at the gate were going to die in production, at several times the price.

What the record shows at scale

Across a portfolio of AI investments, the gate leaves measurable traces:

- Reliance decisions on record, with boundaries, owners, and stop conditions stated.
- Systems scaled within tested case populations, and boundary expansions tied to new evidence.
- Exception-path capacity measured against automation savings before each scale step.
- Fallbacks exercised on schedule, and stop conditions that triggered real pauses.
- Post-launch support consumption against the funded model.
- Production incidents traceable to untested case classes — the number the gate exists to drive down.

The measures also protect the gate from itself. A gate whose latency climbs while its catch rate stays flat is drifting toward ceremony. A portfolio whose boundary expansions have stalled despite accumulating evidence is being governed by caution instead of proof. Both patterns show in the record, and both are correctable — if someone reads the gate's numbers with the same skepticism the gate applies to demos.

The last measure is the outcome. Incidents from untested classes are the signature of demo-driven scaling. Their decline is the gate working. The first measure — written reliance decisions — is the cheapest and most diagnostic: a company that cannot produce the document has not made the decision, whatever its systems are doing in production.

The strongest counterargument

Not every experiment needs production-grade controls. Exploring can be fast and loose when the output cannot cause real harm. The error is letting experiment results approve production reliance without crossing a deliberate gate.

EXHIBIT 2

Keep the gate honest

Proportionality
Depth scales with consequence. Small reversible requests clear in a week.

Clock discipline
Track the gate's own latency as seriously as its catch rate.

Boundary growth
Expansions tie to new evidence, not to accumulated caution.

The outcome measure
Incidents from untested case classes, trending down.

A second objection carries real weight: seven questions can become seven committees, and a gate this thorough can calcify into bureaucracy that teaches teams to route around it. The defense is matching depth to stakes and clock discipline. The gate's depth should scale with consequence. Reversible, low-stakes automations should clear it in a week on thin evidence. And the gate owner should track its own latency as seriously as its catch rate. A gate that cannot say "small yes, quickly" will eventually be unable to say anything at all, because nothing will be brought to it.

What counts as proof

The whole argument comes down to what counts as proof. A demo proves possibility; production reliance has to be earned by operating evidence. The complete workflow, measured. Ownership, accepted. Hostile cases, tested. Exceptions, rehearsed. Controls, visible. Support, funded. A stop that works. Between those two standards sits most of the distance between AI activity and AI value — and unlike model skill, that distance is entirely within a company's control.

For leaders, the working rule fits in a sentence: treat every impressive demo as the first step of an evidence plan. The enthusiasm is fine. It funds the work. The gate just makes sure enthusiasm and evidence arrive at the reliance decision in the right order.

The demos will keep improving. Improving the demos is the vendors' job; building the gate is the company's — and the companies that build it are the ones whose impressive demos grow into systems their operations can actually stand on.

Notes and sources

[1] Alex Singla, Alexander Sukharevsky, Bryce Hall, Lareina Yee, and Michael Chui, with Tara Balakrishnan, "The state of AI in 2025: Agents, innovation, and transformation," McKinsey & Company, November 5, 2025. https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai

[2] Deloitte AI Institute, "The State of AI in the Enterprise," 2026. https://www.deloitte.com/us/en/what-we-do/capabilities/applied-artificial-intelligence/content/state-of-ai-in-the-enterprise.html

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani